

BANKING & FRAUD TRANSACTION SIMULATOR

Build Your Own Project (BYOP) - VITyarthi

Submitted By: **Uddhav Varshney**

Registration Number: **25BAI10140**

Branch: **Computer Science {AI & ML}**

Date: **September 2026**

1. INTRODUCTION

The Banking & Fraud Transaction Simulator is a robust, object-oriented console application developed in Java. It is designed to model core banking operations alongside a real-time rules-based fraud detection mechanism. The system facilitates account creation, fund deposits, balance inquiries, and inter-account transfers. A key focus of this implementation is demonstrating thread-safe concurrency using object-level synchronization, ensuring the integrity of financial data when multiple simulated transactions occur simultaneously.

2. PROBLEM STATEMENT

In standard digital banking architectures, processing concurrent transactions without proper state isolation leads to critical race conditions, commonly referred to as the "double-spending" problem. If multiple threads attempt to withdraw from an account simultaneously, an unsynchronized system might approve transactions that exceed the actual balance. Furthermore, modern banking requires automated oversight to flag anomalous activities immediately. The goal of this project is to build a reliable backend simulator that resolves race conditions using Java's monitor locks and introduces an automated service layer for logging and flagging suspicious transactions.

3. FUNCTIONAL REQUIREMENTS

- **Account Management:** System must allow the creation of unique user accounts with an initial deposit balance and track them via unique integer IDs.
- **Transaction Processing:** Users can deposit funds and transfer amounts between source and destination accounts.
- **Fraud Detection:** Any transaction exceeding a predefined threshold (e.g., \$10,000) must be automatically flagged as suspicious without halting the transaction's success state.
- **Ledger & Statements:** The system must record every transaction (deposits, transfers, failures) with a generated UUID, timestamp, type, and status. It must generate account-specific statements upon request.
- **Fraud Reporting:** An administrative function must exist to extract and display all flagged transactions from the global ledger.

4. NON-FUNCTIONAL REQUIREMENTS

- **Thread Safety (Reliability):** Core monetary modifications (withdrawals, deposits) must utilize synchronization. The system must guarantee that concurrent threads cannot violate balance constraints.
- **Performance:** Data storage should leverage efficient in-memory Java Collections (e.g., `ArrayList`) for rapid $O(n)$ or better read/write operations during simulation.
- **Maintainability:** The application architecture must abide by Object-Oriented principles, strictly decoupling entity models (Accounts, Transactions) from business logic (BankingService, FraudDetectionService).

5. SYSTEM ARCHITECTURE

The system utilizes a modular, multi-tier design encapsulated within a unified Java package:

- **Presentation / Entry Point:** The `BankingSystem` class manages the CLI loop, captures user input via `Scanner`, and triggers appropriate service methods.
- **Service Layer:** `BankingService` acts as the central orchestrator, managing collections of Accounts and Transactions. It delegates rule-checking to the isolated `FraudDetectionService`.
- **Data Model Layer:** Standard POJOs (`Account`, `Transaction`) maintain state variables. State definitions are tightly controlled using Java Enums (`TransactionType`, `TransactionStatus`).

Concurrency Architecture:

Instead of locking the entire service or the collection, the `synchronized` keyword is applied directly to the `deposit()` and `withdraw()` methods inside the `Account` entity. This object-level monitor lock ensures high throughput, as transactions involving different accounts do not block one another.

6. DESIGN DECISIONS & RATIONALE

- **In-Memory Collections:** Java's `ArrayList` was adopted to serve as an ephemeral database for both accounts and the transaction ledger. This allows the simulator to run instantly without external database dependencies.
- **Simulated Latency:** A `Thread.sleep(100)` call is intentionally injected into the withdrawal method. This artificial delay exposes the critical section, proving that without the `synchronized` lock, a race condition would occur during concurrent transfers.
- **UUID Utilization:** Standard `UUID.randomUUID()` is utilized and truncated to generate highly unique, alphanumeric transaction tracking IDs (e.g., TXN4F9B).

7. IMPLEMENTATION DETAILS & EXECUTION

The application relies on Java Threading for its concurrent simulation. The `TransactionThread` class extends `Thread`, allowing the main process to spawn multiple independent transaction requests targeting the exact same account simultaneously. The use of `t1.join()` ensures the main thread pauses until parallel transactions resolve, allowing accurate balance reporting.

Execution Results Snippet

```
===== BANKING & FRAUD TRANSACTION SIMULATOR ===== 1. Create
Account 2. Deposit Funds 4. Transfer Money 7. Concurrent Transaction Demo Enter Choice: 7
===== CONCURRENT TRANSACTION DEMO ===== Creating test account (ID: 999) with $1000
balance... Thread 1 and Thread 2 will attempt to transfer $800 simultaneously. Thread-1
initiating transfer of $800.0... Thread-2 initiating transfer of $800.0... Transfer
completed. Status: SUCCESS Transfer failed: Insufficient funds. Demo Completed. Final state
of Source Account: Account ID: 999 | Holder: Shared Business | Balance: $200.0
```

8. CHALLENGES & KEY LEARNINGS

The primary challenge was orchestrating the multi-threaded demo while avoiding deadlocks and ensuring clear console output. Since standard out is shared across threads, logs occasionally interlaced. By centralizing the transaction logic within the `transfer()` method of the `BankingService` and securing the mutation points in the `Account` object, the architecture naturally protected the ledger's state. This project successfully cemented the practical usage of monitor locks, POJO data structuring, and the separation of concerns through dedicated Service classes.

9. FUTURE ENHANCEMENTS

- **Machine Learning Fraud Detection:** Replace the static threshold (\$10,000) with a predictive model that evaluates transaction velocity, historical averages, and location-based anomalies.
- **Database Persistence:** Migrate the `ArrayList` stores to a SQL database (PostgreSQL/MySQL) leveraging JDBC to persist states across application restarts.
- **Deadlock Prevention Algorithms:** Implement advanced lock ordering when transferring funds between two synchronized accounts to prevent circular wait deadlocks in high-throughput environments.